Underused Fixed-Function GPU Engines as Auxiliary Accelerators for AI Workloads

A Research Agenda for Exploiting Heterogeneous NVIDIA GPU Hardware Beyond Tensor and CUDA Cores

Abstract

Modern NVIDIA GPUs are heterogeneous computing devices containing substantially more specialized hardware than the CUDA and Tensor cores normally associated with artificial-intelligence workloads. Depending on architecture and product, these resources can include dedicated video encoders and decoders (NVENC/NVDEC), an Optical Flow Accelerator (OFA), ray-tracing hardware, copy engines, and, on some embedded platforms, additional vision and deep-learning accelerators.

During many AI workloads, particularly generative inference, some of these fixed-function engines may remain lightly utilized or entirely idle while Tensor and CUDA resources approach saturation.

This paper proposes a research direction: **treat fixed-function GPU engines as auxiliary processors in AI pipelines rather than merely as input/output peripherals.** The objective is not to make fixed-function hardware perform arbitrary computation. Instead, AI algorithms could be reformulated so that useful subproblems resemble operations that these processors already implement efficiently in silicon.

Potential applications include hardware-assisted temporal consistency for generative video, segmentation propagation and part tracking, compressed visual memory, motion-derived conditioning, inference integrity signals, latent or cache storage, and heterogeneous model pipelines in which recognition occurs on Tensor cores while tracking, encoding, decoding, or motion estimation proceeds concurrently on dedicated engines.

Many proposed applications in this paper are speculative and have not been experimentally validated. The purpose is to identify plausible hypotheses and low-cost experiments that could determine whether otherwise-idle fixed-function GPU resources represent an overlooked source of useful AI compute.

---

1. Motivation

AI performance discussions commonly treat an NVIDIA GPU primarily as a collection of CUDA cores, Tensor cores, memory, and bandwidth.

The physical device, however, contains additional specialized processors.

A simplified view is:

Hardware	Primary purpose
Tensor cores	Matrix operations and AI
CUDA/SM cores	General programmable GPU computation
RT cores	Ray traversal and intersection operations
NVENC	Hardware video encoding
NVDEC	Hardware video decoding
Optical Flow Accelerator	Motion/disparity estimation
Copy/DMA engines	Memory movement
DLA, PVA, VIC on some NVIDIA SoCs	Specialized inference, vision and image processing

These units do not necessarily contend for exactly the same execution resources.

For example, NVIDIA describes its Optical Flow Accelerator as operating independently of graphics and CUDA cores, allowing flow computation to be offloaded while CPU and GPU resources remain available for other work. NVIDIA similarly provides dedicated hardware video encode/decode through its Video Codec SDK.

This raises a general systems question:

Can an AI application increase effective total-device utilization by reformulating useful auxiliary computations to run on specialized hardware that would otherwise be idle?

The proposed approach differs from ordinary GPU acceleration. Rather than asking which programmable processor executes an algorithm fastest, it asks:

> Does part of the problem already resemble an algorithm permanently implemented somewhere else on the GPU?

If so, that specialized engine may provide effectively parallel computation without consuming the primary Tensor-core budget of the AI model.

---

2. Fixed-Function Computation Is Not General-Purpose Computation

The central limitation should be stated clearly.

NVENC cannot be programmed to multiply arbitrary matrices. NVDEC cannot execute arbitrary decompression algorithms. OFA cannot perform semantic object recognition.

Their usefulness depends upon **mapping a useful AI subproblem onto the function the hardware already performs**.

This produces a general pattern:

AI problem → compatible representation → fixed-function operation → useful auxiliary signal

Examples might include:

- temporal coherence → optical flow,
- object persistence → motion-vector propagation,
- historical visual state → video compression,
- old reference frames → hardware video decoding,
- spatial correspondence → flow/disparity estimation,
- anomaly signal → residual or motion statistics.

The concept is therefore closer to algorithmic reformulation than hardware repurposing.

---

3. Optical Flow Accelerator: The Most Immediately Applicable Case

Among the hardware considered here, NVIDIA's Optical Flow Accelerator appears to have the clearest near-term AI applications.

Beginning with Turing-generation GPUs, NVIDIA provides dedicated hardware for estimating optical flow between images. It returns motion vectors describing apparent movement between frames while operating independently of CUDA/graphics cores.

Importantly, this is already being used in hybrid AI pipelines.

NVIDIA's OFA tracker architecture decodes video with NVDEC, performs semantic detection using a GPU object detector, and then uses the optical-flow accelerator to assist in tracking the detected objects across subsequent frames. NVIDIA describes this as replacing a computationally intensive feature-extraction/tracking step that would otherwise execute on CPU or CUDA resources.

NVIDIA reports that its optical-flow-based tracker reduced GPU utilization by up to 80% compared with some alternative tracking algorithms in its experiments without compromising tracking accuracy.

This existing architecture suggests several extensions.

3.1 Semantic segmentation propagation

Running a segmentation network on every video frame can be expensive.

Instead:

1. Execute high-quality semantic segmentation on a key frame.
2. Label objects and object parts.
3. Use optical flow to project those masks into subsequent frames.
4. Apply inexpensive refinement.
5. Re-run full segmentation periodically or when tracking confidence declines.

For example:

```
Frame N
    ↓
AI segmentation
    ↓
person
 ├── face
 ├── torso
 ├── left arm
 └── right arm
    ↓
OFA motion field
    ↓
projected masks for N+1
    ↓
lightweight correction
```

The semantic model answers *what the region is*.

OFA helps answer *where that region moved*.

This could be particularly useful for part-level tracking because physical components tend to exhibit coherent or articulated motion even when their appearance changes.

---

4. Generative Video: Hardware-Assisted Temporal Memory

Generative video provides perhaps the most interesting unexplored application.

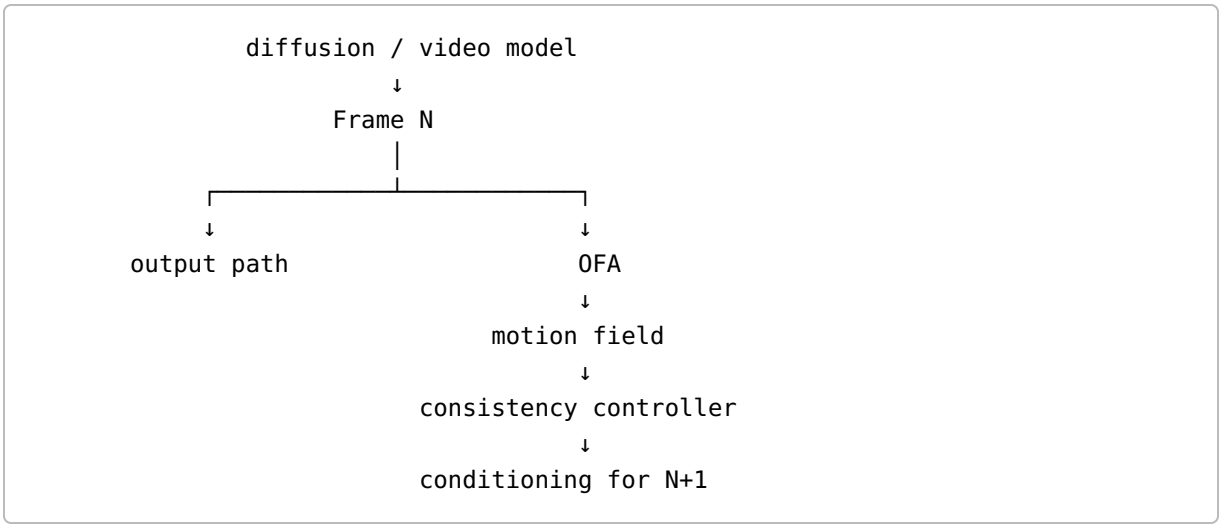
A persistent weakness in generated video is temporal drift. Individual frames may be convincing while identities, textures, clothing, object geometry, backgrounds, hands, or small details change between frames.

A generative system therefore needs both:

1. semantic knowledge of what should exist, and
2. temporal knowledge of how persistent structures have moved.

Optical-flow hardware naturally provides the second.

A possible generation pipeline is:



The model could use motion estimates to constrain the location of known objects and parts during subsequent generation.

Rather than merely conditioning on:

> Maintain the same person.

the system could retain explicit persistent structures:

and propagate their expected locations through OFA-derived flow.

This does not eliminate the need for generative reasoning. Occlusion, newly exposed surfaces, camera changes, deformation and object interaction still require model inference.

It may, however, provide a computationally inexpensive source of **temporal constraint** while the main generative model continues executing.

---

5. NVENC as a Motion-Analysis Engine

NVENC is primarily a video encoder, but NVIDIA exposes a motion-estimation-only mode on supported hardware.

In this mode, the encoder can perform hardware motion searches and return motion vectors and mode information without requiring an ordinary complete video encode. NVIDIA explicitly notes potential use for motion-compensated filtering and custom codecs. For computer vision, AI and frame interpolation, NVIDIA recommends the newer Optical Flow Accelerator because its vectors provide better visual matching.

This distinction suggests a useful hierarchy:

OFA should generally be tested first for semantic tracking and visual correspondence.

NVENC motion estimation may nevertheless be useful where codec-style block correspondence is sufficient, where OFA is unavailable, or where codec-specific information itself is valuable.

A broader research question is whether encoder decisions expose useful low-cost statistics about scene evolution.

For example:

- regions that require large residuals,
- blocks whose prediction suddenly becomes poor,
- changes in reference-frame selection,
- unusual motion-vector discontinuities,
- regions whose compressibility abruptly changes.

Such information could potentially act as an inexpensive **change detector**.

---

6. NVENC/NVDEC as Compressed Visual Memory

Another possibility is to treat hardware video compression as a persistent visual-memory subsystem.

A long-running video-generation or video-analysis system may need access to earlier visual states.

Keeping every raw frame in GPU memory is expensive.

A heterogeneous pipeline could instead maintain:

```
recent frames
    ↓
NVENC
    ↓
compressed historical visual memory
    ↓
VRAM / RAM / storage

when required:

compressed reference
    ↓
NVDEC
    ↓
reconstructed frame
    ↓
model conditioning
```

This is unlikely to outperform learned latent representations when the model already possesses a compact, purpose-built visual state.

However, it has a different advantage:

encoding and decoding are performed by dedicated hardware.

The relevant comparison is therefore not merely compression ratio. Researchers should measure:

- Tensor-core utilization saved,
- CUDA utilization saved,
- VRAM consumption,
- PCIe/memory traffic,
- latency,
- energy consumption,
- reconstruction quality,
- total generated frames per second.

A representation that is mathematically inferior but nearly free on otherwise-idle hardware could still improve whole-system throughput.

Recent NVIDIA tooling also makes this architecture increasingly practical. The current Video Codec SDK supports application-allocated CUDA arrays for NVENC and NVDEC, and NVIDIA specifically highlights reduction of memory copies and CUDA context switches in video pipelines.

---

7. Compression as AI State Storage

The same idea invites experimentation outside literal video.

Arbitrary tensors can theoretically be transformed into image-like arrays and passed through a video codec. This alone does **not** make NVENC a general-purpose tensor compressor.

Nevertheless, some AI states have correlations that compression algorithms may exploit.

A speculative example is transformer KV-cache storage.

Modern research already demonstrates substantial gains from specialized KV-cache compression. Recent work applies transform coding, quantization and entropy coding specifically to transformer state, with reported compression ratios ranging from several-fold to much larger values in particular settings.

A video codec is unlikely to outperform algorithms designed around transformer statistics.

The experiment could nevertheless ask a different question:

> Is a mediocre compression algorithm running almost entirely on an otherwise-idle processor useful at the whole-system level?

Possible mapping dimensions include:

```
X = feature dimension
Y = attention head
T = token position
```

or alternative layouts that cause correlated values to become spatially or temporally adjacent.

The appropriate benchmark would compare:

uncompressed KV,
purpose-built KV quantization/compression, and
fixed-function-assisted storage

while measuring not only accuracy and compression ratio but also Tensor/CUDA occupancy and end-to-end token throughput.

This should be considered a low-probability/high-information experiment rather than a proposed replacement for modern KV compression.

---

8. Independent Integrity and Anomaly Channels

Dedicated engines may also have value as parallel observers of AI state.

Instead of duplicating an expensive computation, an auxiliary engine could continuously generate a cheap structural fingerprint of selected intermediate data.

For video generation, this could include:

- global motion,
- local motion discontinuity,
- residual energy,
- unexpected scene-wide movement,
- persistence of static regions.

A system might learn an expected range for these signals and flag large deviations.

For example:

```
generated frame
      │
      ├── primary model continues
      │
      └── auxiliary hardware analysis
                  ↓
          structural signature
                  ↓
         consistency monitor
```

This would **not** constitute mathematical error correction and would not prove semantic correctness.

Its role would be closer to a low-cost anomaly detector.

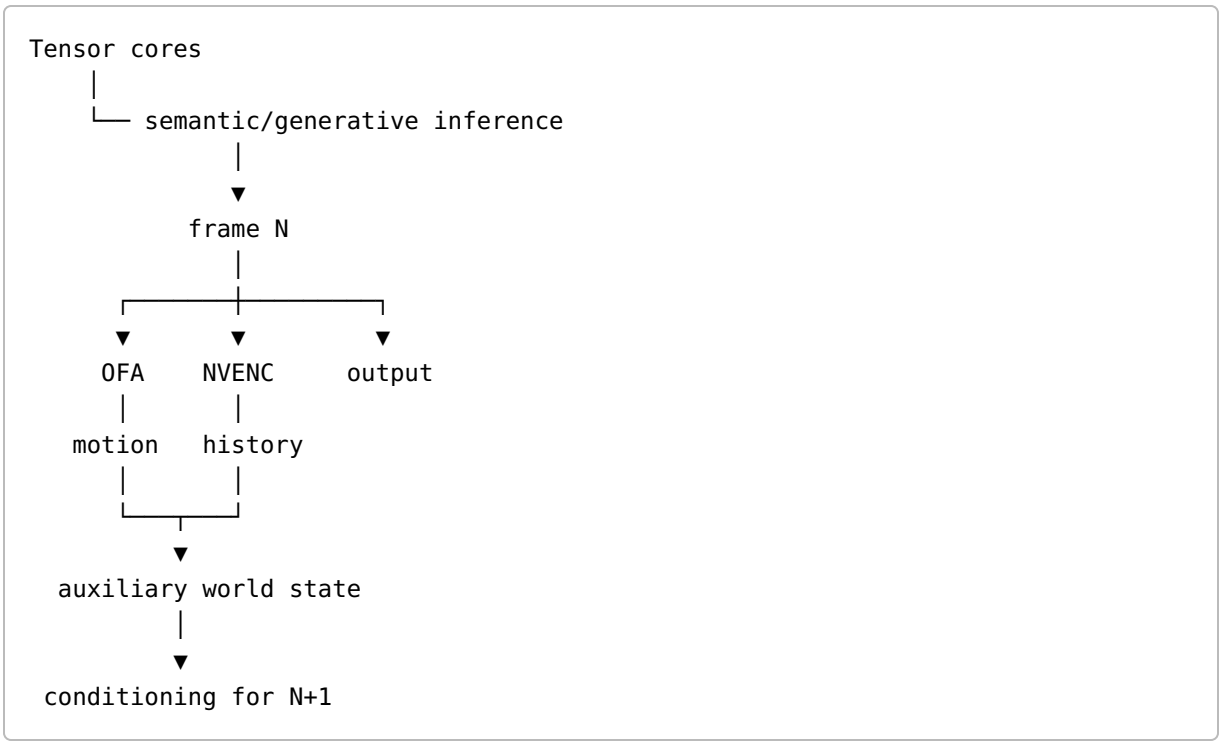
Such a system could potentially identify abrupt temporal corruption, failed intermediate frames, discontinuities, unexpected camera jumps or other coarse failures early enough to trigger regeneration.

---

9. Hardware-Assisted Generative Feedback

The broader architectural possibility is a generative feedback system in which expensive semantic reasoning and inexpensive physical/temporal tracking operate independently.

For video:



This resembles a simple world-state architecture.

The generative network supplies semantic intelligence.

Specialized processors continuously supply inexpensive observations about continuity and motion.

The central research hypothesis is:

> **A weaker auxiliary computation may still improve an AI system if it can run concurrently on hardware that would otherwise contribute nothing to the workload.**

---

10. Image Generation

Single-image generation provides fewer opportunities because temporal coherence does not exist across ordinary isolated images.

Potential applications remain possible for iterative generation:

- tracking spatial structure between intermediate previews,
- detecting large structural changes,
- maintaining segmentation masks,
- hardware-assisted image scaling or transformations on platforms possessing suitable fixed-function blocks.

These seem less compelling than video because existing latent representations, attention maps, feature embeddings, depth maps and segmentation models already provide representations closely matched to image-generation algorithms.

---

11. Audio and Music Generation

NVENC and NVDEC do not provide corresponding audio encoding hardware.

Audio could theoretically be converted into a spectrogram or similar image representation and processed indirectly, but doing so introduces conversion overhead and maps poorly onto the underlying task.

Learned audio codecs, audio tokens, spectral features and model-native latent representations appear substantially more appropriate.

Consequently, music generation seems unlikely to benefit directly from NVENC/NVDEC.

The broader heterogeneous-compute principle could still apply if future or platform-specific hardware exposes suitable DSP or audio accelerators.

---

12. Beyond Desktop RTX

The concept becomes broader on NVIDIA embedded systems.

NVIDIA Jetson platforms expose additional specialized accelerators. NVIDIA describes its Deep Learning Accelerator as fixed-function hardware for neural-network inference, while newer Jetson systems also expose processors for vision and image-processing tasks.

For example, NVIDIA describes the Vision Image Compositor on Jetson as specialized hardware for operations including rescaling, remapping, warping, color conversion and noise reduction.

This suggests an even broader research agenda:

> Given an AI pipeline and a heterogeneous SoC/GPU, automatically determine which subgraphs can be executed on specialized processors without reducing model quality.

Such scheduling resembles compiler optimization across fundamentally different processors.

---

13. Proposed Experimental Program

The ideas above could be evaluated without initially constructing a new foundation model.

Experiment 1: Segmentation propagation

Run a modern segmentation model at different intervals:

- every frame,
- every 2 frames,
- every 5 frames,
- every 10 frames.

Between semantic passes, propagate masks using OFA.

Measure:

- segmentation accuracy,
- part-ID stability,
- Tensor/CUDA utilization,
- GPU power,
- throughput.

Hypothesis: OFA-assisted propagation can reduce semantic inference frequency while retaining acceptable temporal accuracy.

Experiment 2: Generative-video consistency

Generate identical prompts with:

A. normal model execution,

B. optical-flow-derived mask or motion conditioning,

C. software optical-flow conditioning,

D. hardware OFA conditioning.

Measure:

- temporal consistency,
- identity preservation,
- part persistence,
- inference speed,
- GPU utilization,
- power.

The critical comparison is not simply OFA versus learned optical flow.

It is **whole-system performance when the primary GPU is already compute-bound**.

Experiment 3: Compressed video memory

Maintain long historical frame sequences using:

- raw frames,
- reduced-resolution frames,
- latent representations,
- NVENC H.264/HEVC/AV1 frames.

Retrieve historical references through NVDEC.

Measure:

- VRAM,
- memory bandwidth,
- reference latency,
- reconstruction quality,
- generation consistency.

Experiment 4: NVENC motion vectors versus OFA

Compare:

- NVENC ME-only motion vectors,
- NVOFA flow,
- software optical flow,
- learned optical flow.

Test their usefulness for:

- object tracking,
- part tracking,
- mask propagation,
- generative temporal conditioning.

This would establish whether lower-quality but inexpensive codec motion vectors are still sufficient for particular AI-side tasks.

Experiment 5: Structural anomaly detection

Generate motion/residual statistics from otherwise-valid video and deliberately corrupted sequences.

Determine whether fixed-function signals reliably identify:

- frame replacement,
- object disappearance,
- identity discontinuity,
- unexpected static-background movement,
- severe generative artifacts.

Experiment 6: Non-video tensor compression

Map KV-cache or activation tensors into several 2D/temporal layouts.

Measure video-codec compression and reconstruction effects.

This experiment should be expected to fail against purpose-built tensor compression.

Its purpose is to determine whether **concurrency and specialized-hardware availability compensate for inferior compression characteristics**.

A negative result would still establish a useful boundary for the broader hypothesis.

---

14. Evaluation Must Measure the Whole GPU

A major risk in this research direction is optimizing the wrong metric.

For example, an OFA operation may be slower than a CUDA kernel in isolation but still improve system performance if the CUDA cores are needed simultaneously for model inference.

Accordingly, experiments should report:

- end-to-end throughput,
- latency,
- Tensor-core occupancy,
- CUDA/SM occupancy,
- fixed-function utilization,
- memory bandwidth,
- VRAM consumption,
- CPU utilization,
- energy per generated token/frame,
- model-quality changes.

The question is not:

> Which processor performs this operation fastest?

It is:

> Which allocation produces the greatest useful output from the entire device?

---

15. Limitations and Reasons the Approach May Fail

Several factors could make these ideas impractical.

Data conversion overhead

If large CUDA kernels are necessary merely to convert model state into a fixed-function-compatible representation, the cost may erase the benefit.

Memory bandwidth

Separate processors do not imply separate memory systems. Concurrent operations can still contend for VRAM bandwidth and caches.

Synchronization

Generative models may have strict dependency chains. If the Tensor pipeline must wait for OFA, NVENC or NVDEC, concurrency may disappear.

Representation mismatch

A video codec's notion of similarity is optimized for video compression, not neural-network state.

Existing learned representations

Modern AI models already employ highly optimized latent representations. Fixed-function alternatives may simply provide less useful information.

API restrictions

Hardware capabilities vary by GPU generation and driver/API support.

Better programmable alternatives

A CUDA implementation may be sufficiently inexpensive that adding another subsystem is not worthwhile.

These limitations are exactly why empirical evaluation is necessary.

---

16. Research Opportunity

The individual ideas in this paper are less important than the underlying systems question.

Modern AI workloads increasingly treat GPUs as matrix processors with memory attached.

Modern GPUs are actually collections of heterogeneous processors.

At the same time, AI workloads are becoming increasingly multimodal and stateful. Video models need motion and temporal memory. Vision systems need tracking. Agents need persistent memory. Generative systems need consistency checks. Long-context inference needs increasingly sophisticated storage hierarchies.

Some of these auxiliary tasks bear striking resemblance to operations that specialized GPU hardware already performs.

The opportunity is therefore to investigate **heterogeneous AI inference that deliberately co-schedules semantic computation and fixed-function computation**.

A future inference engine might regard a GPU not as:

```
AI model → Tensor cores
```

but as:

```
                    ┌─ Tensor cores → model inference
                    ├─ CUDA cores → general kernels
AI computation graph ├─ OFA → motion/state propagation
                    ├─ NVENC → compressed visual history
                    ├─ NVDEC → historical-state retrieval
                    ├─ RT hardware → spatial queries
                    └─ platform accelerators → vision/image operations
```

Compiler/runtime research could eventually determine whether subproblems can be mapped automatically onto available hardware.

---

17. Conclusion

This paper does not claim that NVENC, NVDEC or NVIDIA's Optical Flow Accelerator provide undiscovered general-purpose compute.

They do not.

Instead, it proposes a narrower hypothesis:

AI systems may leave useful computation capacity unused because algorithms are normally designed around programmable CPU/GPU resources rather than around the complete collection of specialized processors physically present on the device.

The clearest existing evidence is the Optical Flow Accelerator. NVIDIA already demonstrates AI pipelines in which object detection occurs on conventional GPU resources while a dedicated flow engine performs tracking work and frees those resources for other tasks.

Generative video appears especially suited to extending this principle because motion tracking, temporal consistency, reference-frame retrieval and compressed history align naturally with video-oriented fixed-function hardware.

Other ideas—particularly KV-cache compression and generic tensor processing through video codecs—are substantially more speculative and may prove ineffective. They nevertheless offer inexpensive experiments capable of defining where this strategy does and does not work.

The broader research question is therefore worth testing:

> **How much additional useful AI computation can be extracted from a modern GPU by designing algorithms around the entire heterogeneous chip rather than only its programmable compute cores?**

The answer may be “very little.”

But given that much of the relevant silicon already exists, is already powered, and may already be idle during expensive AI inference, finding out could be worthwhile.